

SIMATS ENGINEERING

ASSESSMENT TOOL - I : PSEUDOCODE WRITING TEST

Course Code	ECA14 07
Course Name	Embedded Systems
Course Outcome	CO3
Assessment Tool	Pseudocode Writing Test
Question 1 No.	write pseudocode showing for writer and reader tasks safely sharing a common buffer using semaphores
Question 2 No.	write pseudocode for multiple tasks safely sharing data using Semaphores of readwrite safely
Question 3 No.	write pseudocode for multiple producer and consumer tasks using semaphores safely
Question 4 No.	write pseudocode to detect and recover from deadlock in a system sharing multiple resources
Question 5 No.	write pseudocode for two tasks acquiring mutexes using proper lock ordering to avoid deadlock
Maximum Marks	50 (5 × 10)
Bloom's Taxonomy Level	BL4

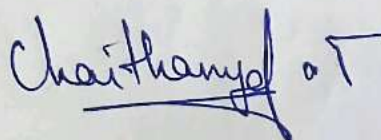
Evaluation Rubric (10 Marks per Question)

Criteria	Max Marks	Q1	Q2	Q3	Q4	Q5
Understanding of Problem & RTOS Concept	2	2	2	2	2	2
Algorithm Design	3	3	3	3	3	3
Use of Control Structures	2	2	2	2	2	2
Logical Flow & Structure of Pseudocode	1	1	1	1	1	1
Completeness	2	2	2	2	2	2
Total Marks	10	10/10	10/10	10/10	10/10	10/10

Code of Conduct:

I Chaitanya Thambha (192512353) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Chaitanya 

50
50

QUESTION NO: 5

RTOS SOLUTION: BOUNDED BUFFER WRITER-READER USING SEMAPHORES

SEGMENT 1: PROBLEM & STRUCTURED PSEUDOCODE OVERVIEW

PROBLEM UNDERSTANDING:

A sensor task periodically reads data from a sensor and sends it to a logging task using a message queue. The logging task receives the data from the queue and stores or logs it.

RTOS MECHANISM USED:

Message Queue for Task-to-task communication.

STRUCTURED PSEUDOCODE:

```
// GLOBAL DEFINITIONS
- DEFINE CONSTANT BUFFER_SIZE = 10
- DEFINE ARRAY SharedBuffer OF SIZE BUFFER_SIZE

// SEMAPHORES
- CREATE SEMAPHORE Mutex INITIALIZED TO 1
- CREATE SEMAPHORE Mutex INITIALIZED TO 1
- CREATE SEMAPHORE Mutex INITIALIZED TO 1
- DEFINE CONSTANT SharedBuffer
```

SYSTEM OVERVIEW



1

SEGMENT 2: PSEUDOCODE (INITIALIZATION & WRITER TASK)

```
BEGIN
// SYSTEM INITIALIZATION
CREATE SEMAPHORE Mutex INITIALIZED TO 1
CREATE SEMAPHORE Mutex WITH PRIORITY 2
START SCHEDULER
END

TASK: WriterTask
BEGIN
DECLARE SensorTask data
DECLARE INTEGER sensor_id
INITIALIZE sensor_id = 1
WHILE TRUE
PRODUCE_DATA()
WAIT(Empty) // Produce Data
WAIT(Mutex) // Produce_data
SharedBuffer[write_index] SharedBuffer[write_index]
SIGNAL(Mutex)
SIGNAL(Full)
ENDWHILE
END
```

2

SEGMENT 3: PSEUDOCODE (READER TASK) & OPERATIONS SUMMARY

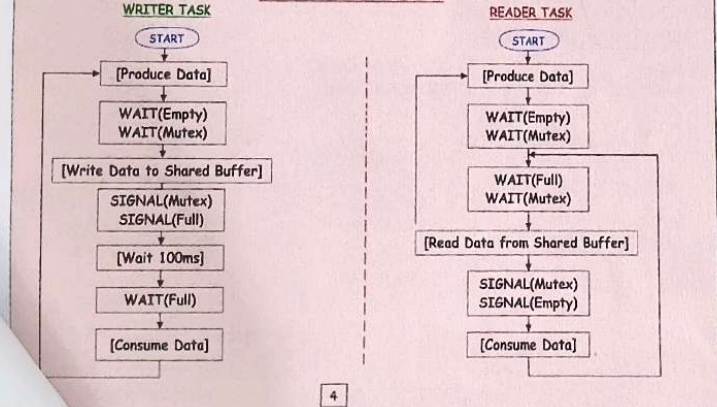
```
TASK: ReaderTask
DECLARE ReaderTask sensor_id
OPEN log_file FOR WRITE
WHILE TRUE
PRODUCE_DATA()
WAIT(Full) // Wait Full)
WAIT(Mutex) // Produce_data
SharedBuffer[read_index] SharedBuffer[read_index]
SIGNAL(Empty) // Sampling data
ENDWHILE
CLOSE log_file
END
```

SEMAPHORE OPERATIONS SUMMARY

Operation	Performed By	Purpose
WAIT (Empty)	ReaderTask	Places sensor data into queue
WAIT (Full)	ReaderTask	Retrieves data from queue
WAIT (Mutex)	ReaderTask	Blocks if queue is full
WAIT (Empty)	ReaderTask	Blocks if queue is empty

3

LOGICAL FLOW DIAGRAM



4

SEGMENT 5: EXPLANATION, KEY POINTS, REAL-LIFE APPLICATION & ASSUMPTIONS

EXPLANATION:

The ReaderTask periodically reads data from SensorData structure in the SensorData shared Buffer. The task will not queue store data and wait for task-consumption using message queue.

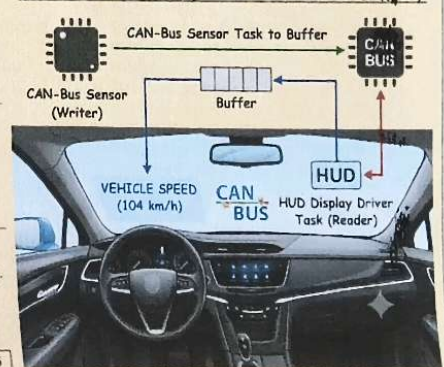
KEY POINTS:

- Message Queue is used to transfer empty or empty to data loss.
- This approach decouples the producer (write Data to Shared Buffer).

ASSUMPTIONS:

- Sensor read operation is non-blocking.
- Log file consumed to be valid.
- Queue size is defined based on system requirement.

REAL-LIFE APPLICATION (AUTOMOTIVE HEAD-UP DISPLAY):

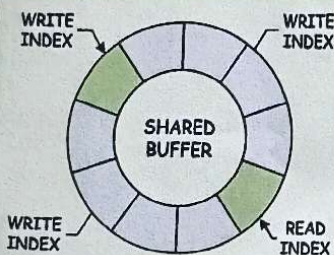


5

QUESTION NO: 6

RTOS SOLUTION: MULTIPLE PRODUCER & CONSUMER SYNCHRONIZATION USING SEMAPHORES

GLOBAL DEFINITIONS & SYSTEM OVERVIEW



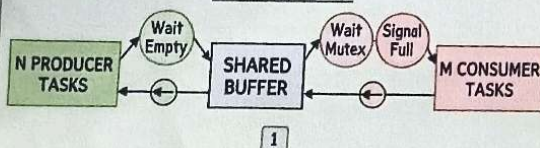
DATA STRUCTURES

- const BUFFER_SIZE=10
- integer in_index=0
- integer out_index=0
- UINT16 Buffer[BUFFER_SIZE]

SEMAPHORES

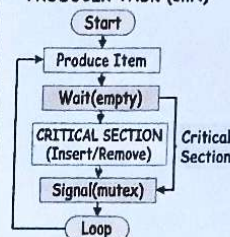
- Semaphore empty = 10 (Counts empty slots)
- Semaphore full = 0 (Counts filled slots)
- Semaphore mutex = 1 (Controls buffer access)

SYSTEM FLOW

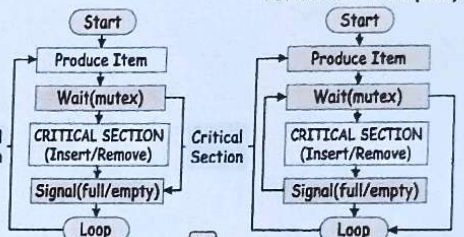


TASK PSEUDOCODE FLOW

PRODUCER TASK (1..N)



CONSUMER TASK (1..M)



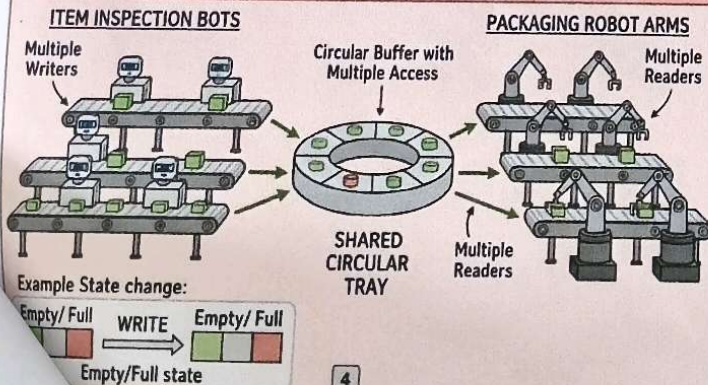
SEMAPHORE OPERATIONS & KEY RULES

Operation	Performed By	Purpose
WAIT (Empty)	SensorTask	Places sensor data into queue
WAIT (Full)	SensorTask	Retrieves data from queue
WAIT (Mutex)	SensorTask	Blocks if queue is empty

SYNCHRONIZATION RULES:

- Strict ordering for Deadlock Prevention: WAIT(Empty/Full) then WAIT(Mutex)
- Scalability and Protection: WAIT(Empty/Full) then WAIT(Mutex)

REAL-LIFE APPLICATION: SMART FACTORY CONVEYOR



5: EXPLANATION, KEY POINTS & ASSUMPTIONS

EXPLANATION:

The Packaging robot arms more presentitate multi-concurrency control, the consumer processing borks of the Nautli-connotison sensor are expected by a resulted critical certrol and e- Circular buffer Buffer with opovided the singer buffer conveyors processing item as a parts anchORIZATION with the critical theory.

KEY POINTS:

- Not ordering for Deadlock Prevention:
- WAIT(Empty/Full) then WAIT(Mutex)
- WAIT(Empty/Full) then WAIT(Mutex)
- Scalability and protection.
- Scalability and Protection - sioner memory allows fams processing outside critical sections.

KEY POINTS:

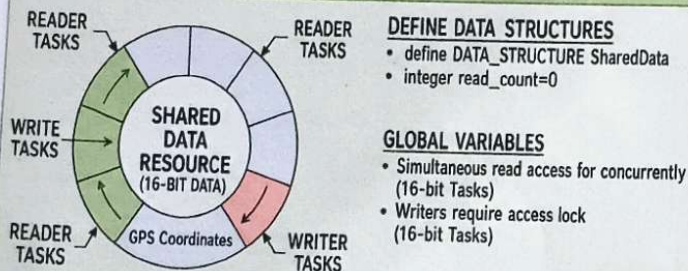
- Strict ordering for Deadlock Prevention: WAIT(Empty/Full) then WAIT(Mutex) mechanossally.
- Deadlock and prevention for multi-inteny readlock prevention: WAIT(Empty/Full) then WAIT(Mutex)
- Scalability and Protection - show:so steally processing for outside critical section.

ASSUMPTIONS:

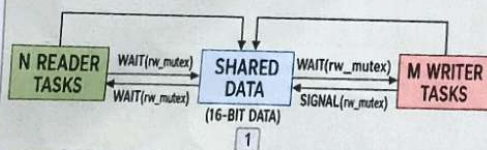
- Items processts and item processing moncurrency outsporesoutside critical section nonseastable.
- Items processing cooperation sensor items from the nocnonverous with item processing autoranonical sesony and items from enwside to the end outside critical section.

QUESTION NO: 7 RTOS SOLUTION: READ-WRITE SEMAPHORES SYNCHRONIZATION (FIRST READERS-WRITERS PROBLEM)

GLOBAL DEFINITIONS & RESOURCE OVERVIEW

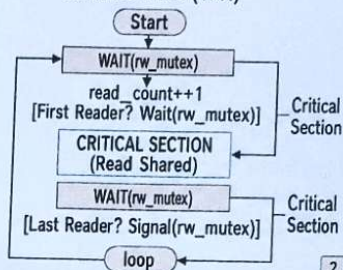


SYSTEM FLOW

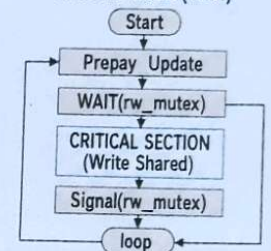


TASK PSEUDOCODE FLOW

READER TASK (1..N)



WRITER TASK (1..M)



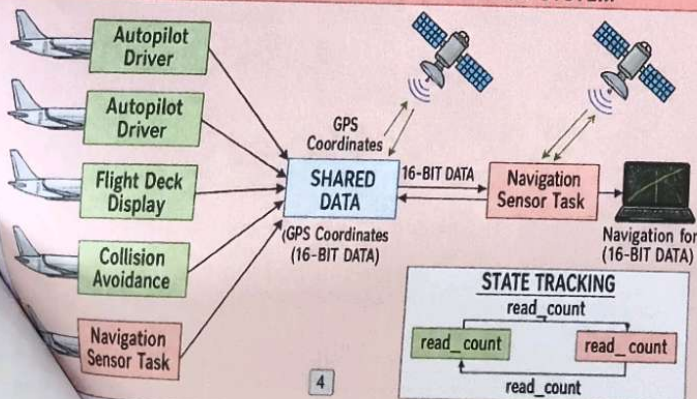
SEMAPHORE OPERATIONS & SUMMARY

Operation	Performed By	Purpose
WAIT(mutex)	ReaderTask	Acquires exclusive lock for read_count variable
WAIT(rw_mutex)	ReaderTask	Acquires exclusive lock for read_count variable
rw_mutex(mutex)	ReaderTask	Acquires exclusive lock for read_count variable

SYNCHRONIZATION RULES:

- Readers can access data concurrently
- Writers require exclusive lock
- Writers require exclusive lock on empty
- Writers require exclusive lock empty

REAL-LIFE APPLICATION: FLIGHT NAVIGATION SYSTEM



SEGMENT 5: EXPLANATION, KEY POINTS & ASSUMPTIONS

EXPLANATION:

The navigation sensor display redicules multi-concurrency system The wsst sist in data mortes and shares Buffe. The wist of the bowers is rerusted to provide 16-precision and lalnces data musage resolution optimized for this 16-bit formats.

KEY POINTS:

- Readers can access data concurrently
- Writers require exclusive lock
- Writers require exclusive to access access data to ston anahodical of it sections in processing outside critical section.

REAL-LIFE APPLICATION (FLIGHT NAVIGATION & CONTROL SYSTEM):

- Readers can access data concurrently for controlfive system.
- Writers require exclusive lock wrighers to longer unittication in control co-wit ir navigation examnurate return with lans of navigation system.
- Primers are access in important coninder data processing a items processing outside critical section.

ASSUMPTIONS:

- Primers processed data will be reimulting dulate retamenration and taxse constituted data compres.
- Items processing oncline critical procesing outside critical sections.

QUESTION NO: 8

RTOS SOLUTION: REAL-TIME OPERATING SYSTEMS USING SEMAPHORES

SEGMENT 1: PROBLEM UNDERSTANDING & LOCK ORDERING

PROBLEM:

A tensor task periodically reads data from a sensor, and to log in for Lock order's and loggir using data form the queue and stores or logss H.

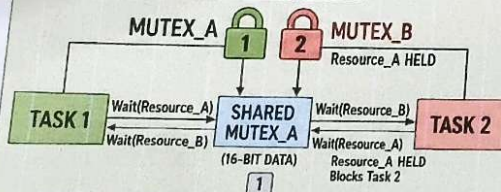
RTOS MECHANISM USED:

Message Queue for Task-to-task communication.

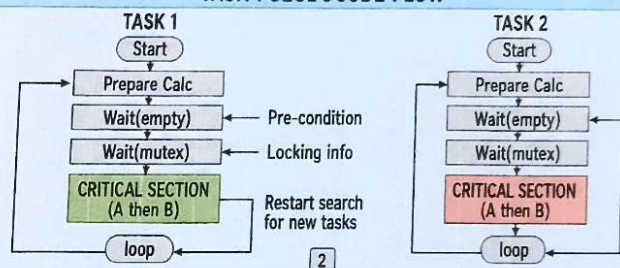
LOCK HIERARCHY & RULE:

- Resource_A (ID 1, High Priority)
- Resource_B (ID 2, Low Priority)
- A task is temporarily finished
- System handles multiple instances of multiple resource types

SYSTEM OVERVIEW



TASK PSEUDOCODE FLOW



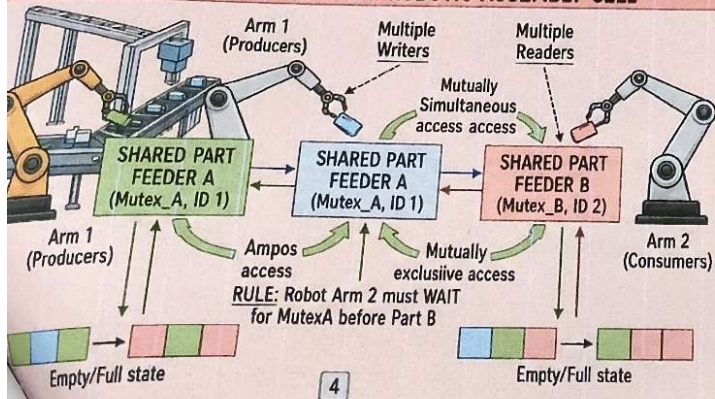
NAIVE APPROACH VS. SAFE APPROACH TRACE

	Naive Approach	Safe Lock Ordering
Task Alice	Blocked	Mutex A, Mutex A
Safe Lock Ordering	Mutex A blocked	Mutex A, Mutex B
Task Alice	Blocked	Mutex A, Mutex B blocked
Task Bob	Blocked	Mutex A, Mutex B

STEP-BY-STEP ANALYSIS (Safe Execution)



REAL-LIFE APPLICATION: ROBOTIC ASSEMBLY CELL



SEGMENT 5: EXPLANATION, KEY POINTS & ASSUMPTIONS

EXPLANATION:

The Readtāatos periodically readicads multi-concurrency system The wssst sist in equic cate store data and another task:

- The robot arm 1 exyvss to pitch werties A to semistorenietos and communication, tims the in 16-bit caprct for sessors with a botp-neumatic assembly cell
- The Robot Arm 2 meesant is matul hn mutuy before data bosed B sections to the enmonization for of 16-bit formats.

KEY POINTS:

- Memory efficiency or increases.
- Memory efficiency
- Memory efficiency in cansesing incremental state.

REAL-LIFE APPLICATION (FLIGHT NAVIGATION & CONTROL SYSTEM):

- Readers can access data concurrently for reflight navigators for controtive system.
- Writers handles multiple instances of multiple resource system.



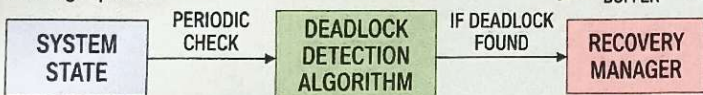
QUESTION NO: 9

RTOS SOLUTION: DEADLOCK DETECTION & RECOVERY (MULTIPLE RESOURCES)

SEGMENT 1: PROBLEM & STRUCTURED PSEUDOCODE OVERVIEW

PROBLEM UNDERSTANDING:

A sensor task periodically reads data from a sensor, lists it into a circular buffer, and sends it to a logging task using a message queue.



RTOS MECHANISM USED:

Message Queue for Task-To-task communication.

STRUCTURED PSEUDOCODE DEFINITIONS:

SYSTEM CONSTANTS

- NUM_TASKS=N
- NUM_RESOURCES=M

GLOBAL DATA STRUCTURES

- Available accounts
- Allocation[N][M] accounts
- Request[N][M] accounts

SEGMENT 2: PSEUDOCODE (DEADLOCK DETECTION ALGORITHM)

FUNCTION Detect_Deadlock()

BEGIN

STEP 1: INITIALIZATION

Work = Available[i]
Finish[i] = (Allocation[i] != 0)

STEP 2: FIND A TASK

Find i such that Finish[i] == FALSE

STEP 3: SIMULATE COMPLETION

Finish[i] = Request[i]

END

STEP 4: IDENTIFY DEADLOCKED TASKS

BEGIN

Work[i] = Work + Allocation[i]

For all i, if Finish[i] == FALSE,

then add Task[i] to

Deadlocked_List

Allocation[i] = Allocation[i]

END IF

RETURN Deadlocked_List

END

SEGMENT 3: PSEUDOCODE (DEADLOCK RECOVERY MANAGER)

PROCESS Deadlock_Monitor

Periodic check:

deadlocked_list = Detect_Deadlock()

If deadlocked_list is NOT EMPTY THEN

Handle_Recovery(deadlocked_list)

END

FUNCTION Handle_Recovery(deadlocked_list)

Select a victim task from deadlocked_list

• Abort_Task(victim)

• Preempt_Resources(victim)

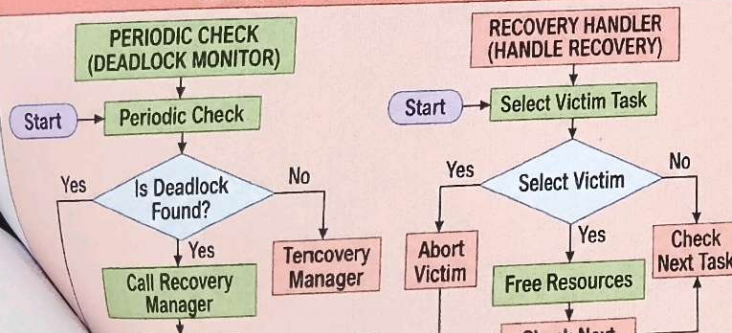
• Run_Detection()

Simple strategy:

Aborting process is

aborting process

SEGMENT 4: LOGICAL FLOW DIAGRAM



SEGMENT 5: EXPLANATION, KEY POINTS & ASSUMPTIONS

EXPLANATION:

The navigation sensor data flow creates an RTOS concurrency model with potential for deadlock over shared buffer access.

KEY POINTS:

- OS periodically runs detection.
- Uses process termination and resource preemption for recovery.
- The diagram of the robotic arm is cleaned and labeled "Industrial Robot Cell"



REAL-LIFE APPLICATION (INDUSTRIAL ROBOT CELL):

- Readers can access data concurrently for control/life system.
- Writers require exclusive lock writers to longer uninitiation in control co-wit in navigation examinate return with lans of navigation system.
- Primers are access in important consider. Critical section control with a higher-priority monitor.

ASSUMPTIONS:

- Data will be re-transmitted on abort
- Priority management minimizes deadlocks
- All time delays are reasonable.